

Bonnes pratiques Git

Alexis VO – CentraleSupélec

1. Conventions de commit – Conventional Commits

Un message de commit clair permet de comprendre rapidement l'historique du projet et facilite notamment l'automatisation des changelogs.

1.1 Format standard

```
<type>(<portee optionnelle>): <description courte a l'imperatif>

[Corps optionnel : expliquer le pourquoi et non seulement le comment ]

[Pied de page optionnel : BREAKING CHANGE: <details> ou Closes #<issue>]
```

1.2 Types de commits principaux

- **feat** : ajout d'une nouvelle fonctionnalité;
- **fix** : correction d'un bug;
- **docs** : modification de la documentation;
- **refactor** : modification interne du code sans ajout de fonctionnalité ni correction de bug;
- **test** : ajout ou modification de tests;
- **chore** : tâches techniques diverses : dépendances, configuration, outils, CI/CD, etc.

1.3 Exemples

```
# Ajout d'une fonctionnalite avec portee
git commit -m "feat(auth): ajout de l'authentification a deux facteurs"

# Correction d'un bug
git commit -m "fix(cart): correction du calcul de la TVA"

# Modification de la documentation
git commit -m "docs: mise a jour du guide d'installation"
```

Bon principe : un commit doit représenter une modification cohérente et idéalement atomique. Le message doit décrire *ce qui a changé* de manière concise.

2. Gestion des branches et workflow

Deux workflows sont couramment rencontrés :

- **GitHub Flow** : une branche principale stable (**main**) et des branches courtes dédiées aux fonctionnalités;
- **Git Flow** : organisation plus structurée avec notamment **main**, **develop**, **feature/*** et éventuellement **release/*** ou **hotfix/***.

Le choix dépend du projet et de l'équipe. Pour de nombreux projets modernes, GitHub Flow est suffisant.

2.1 Stratégie de branches

- **main** : branche principale contenant une version stable du projet;
- **develop** : branche d'intégration utilisée dans certains workflows de type Git Flow;

- `feature/*` : branche dédiée à une nouvelle fonctionnalité ;
- `bugfix/*` : branche dédiée à la correction d'un bug.

Exemples :

```
feature/login-oauth
feature/data-loader
bugfix/header-overflow
bugfix/calculation-error
```

2.2 Commandes essentielles

```
# Créer une nouvelle branche et basculer dessus
git switch -c feature/ma-fonctionnalite

# Ancienne syntaxe equivalente
git checkout -b feature/ma-fonctionnalite

# Afficher les branches
git branch

# Basculer sur une branche existante
git switch main

# Recuperer les modifications du depot distant
git fetch origin

# Mettre a jour la branche courante
git pull

# Mettre a jour depuis main avec un rebase
git pull --rebase origin main
```

Attention : avant un `pull -rebase`, il est préférable d'avoir un arbre de travail propre. Le rebase réécrit l'historique des commits concernés.

3. Nettoyage et réécriture de l'historique

Avant de publier une branche, il peut être utile de nettoyer l'historique local afin d'éviter une succession de commits tels que `wip`, `correction`, `oups`, etc.

3.1 Squash de commits

Le rebase interactif permet de regrouper plusieurs commits en un historique plus cohérent.

```
# Regrouper les 4 derniers commits
git rebase -i HEAD~4
```

Dans l'éditeur :

- conserver `pick` pour le premier commit ;
- remplacer `pick` par `squash` (ou `s`) pour les commits à fusionner ;
- modifier ensuite le message du commit résultant si nécessaire.

3.2 Réécriture de l'historique : précautions

Un rebase modifie les identifiants des commits concernés. Il faut donc éviter de réécrire l'historique d'une branche sur laquelle d'autres personnes travaillent déjà.

Après un rebase d'une branche déjà publiée, on peut devoir utiliser :

```
git push --force-with-lease
```

Préférer `-force-with-lease` à `-force` : il vérifie qu’aucune nouvelle modification distante n’a été publiée depuis la dernière synchronisation connue.

4. Annulations et corrections ciblées

4.1 Annuler les modifications d’un fichier

Pour supprimer les modifications locales non indexées d’un fichier :

```
git restore nom_de_fichier
```

Attention : cette commande supprime les modifications non enregistrées dans un commit.

4.2 Désindexer un fichier

Pour retirer un fichier du *staging area* tout en conservant ses modifications :

```
git restore --staged nom_de_fichier
```

4.3 Annuler le dernier commit

Pour supprimer le dernier commit tout en conservant les modifications dans le répertoire de travail et l’index :

```
git reset --soft HEAD~1
```

Pour supprimer le commit tout en conservant les modifications dans le répertoire de travail, mais en les désindexant :

```
git reset --mixed HEAD~1
```

À retenir : les commandes **reset** peuvent réécrire l’historique local. Elles doivent être utilisées avec précaution sur des commits déjà publiés.

5. Bonnes pratiques et pièges à éviter

⚠ À éviter

- **Ne pas utiliser `git push -force`** sur une branche partagée sans coordination avec l’équipe.
- **Ne jamais versionner de secrets** : mots de passe, clés API, certificats privés ou fichiers contenant des informations sensibles.
- **Éviter les messages de commit vagues** : `fix`, `update`, `correction`, `asdf`, etc.
- **Éviter les commits gigantesques** qui mélangent plusieurs fonctionnalités ou corrections indépendantes.

5.1 Règles d’or pour la collaboration

1. **Commiter régulièrement et de manière atomique** : chaque commit doit correspondre autant que possible à une modification cohérente et compréhensible.
2. **Synchroniser régulièrement sa branche** : récupérer les modifications distantes permet de limiter les conflits importants lors de l’intégration.
3. **Utiliser le rebase avec discernement** : `git pull -rebase` peut maintenir un historique linéaire, mais il ne faut pas réécrire l’historique partagé sans coordination.
4. **Utiliser des Pull Requests** : une PR permet de faire relire le code, de discuter les changements et de conserver une trace claire de la modification.

5. **Rédiger des PR explicites** : décrire le problème traité, les changements effectués, les éventuels impacts et, si nécessaire, ajouter des captures d'écran ou des résultats de tests.
6. **Vérifier avant de pousser** : utiliser notamment `git status`, les tests et, si nécessaire, `git diff` avant le commit et le push.

6. Workflow recommandé

Pour une fonctionnalité classique, un workflow simple peut être :

```
# 1. Se placer sur main et recuperer les dernieres modifications
git switch main
git pull --rebase origin main

# 2. Creer une branche de travail
git switch -c feature/ma-fonctionnalite

# 3. Developper puis verifier l'etat du depot
git status
git diff

# 4. Indexer les modifications
git add .

# 5. Creer un commit atomique
git commit -m "feat: ajout de ma fonctionnalite"

# 6. Publier la branche
git push -u origin feature/ma-fonctionnalite

# 7. Ouvrir une Pull Request

# 8. Apres validation et fusion, supprimer la branche si necessaire
git switch main
git pull --rebase origin main
git branch -d feature/ma-fonctionnalite
```

Résumé

- **Commits** : courts, atomiques et explicites.
- **Branches** : une branche par fonctionnalité ou correction.
- **Historique** : propre avant publication lorsque cela est possible.
- **Collaboration** : Pull Requests et revue de code.
- **Sécurité** : aucun secret dans Git.
- **Force push** : éviter sur les branches partagées; préférer `-force-with-lease` lorsque la réécriture est justifiée.